01 한눈에 보기

매장 숫자를 정리하는 계산 담당

조회할 매장과 달을 받으면 장부에서 필요한 기록을 읽고, 카드와 그래프에 넣을 숫자를 만듭니다.

1. 받아요

컨트롤러가 매장 번호와 조회할 달을 줍니다.

2. 계산해요

저장소가 꺼낸 숫자를 더하고 그래프 순서대로 놓습니다.

3. 돌려줘요

계산 결과를 Summary라는 결과 상자에 담습니다.

앱이 시작할 때 준비하는 것

```
@Service
public class AnalyticsService {
    private final AnalyticsRepository repository;

    public AnalyticsService(AnalyticsRepository repository) {
        this.repository = repository;
    }
```

클래스는 계산 담당의 설계도입니다. @Service를 본 Spring이 이 설계도로 객체를 만들고, 장부를 읽는 Repository 객체를 생성자의 매개변수로 넣어 줍니다. this.repository는 그 객체를 보관하는 필드입니다. private은 클래스 안에서 쓰고, final은 보관한 참조를 다시 바꾸지 못하게 합니다.

파일 위쪽의 이름표

package / import	package는 코드가 속한 묶음입니다. import는 다른 묶음의 타입을 짧은 이름으로 쓰게 합니다.
Service / Transactional	Spring이 관리할 서비스와 DB 작업의 범위를 표시하는 애너테이션입니다.
YearMonth / ChronoUnit / List	연도·월을 담는 클래스, 시간 단위를 고르는 Enum, 여러 값을 순서대로 담는 인터페이스입니다.
Javadoc	역할, 작성자, 변경 이유를 읽는 사람에게 알려줍니다. @param은 입력 설명이고 @return은 반환값 설명입니다.

02 요약 카드

한 달 동안 손님이 얼마나 왔을까?

summarize()는 선택한 한 달의 매출, 방문 건수, 방문 고객 수, 재방문 고객 수를 모읍니다.

1. 매장과 월을 받아 조회 범위를 정해요

```
@Transactional(readOnly = true)
public Summary summarize(long storeId, YearMonth month) {
    var start = month.atDay(1).atStartOfDay();
    var end = month.plusMonths(1).atDay(1).atStartOfDay();
    var customers = repository.summarizeByCustomer(storeId, start, end);
```

storeId는 매장 번호를 받는 long 매개변수이고, month는 월 객체를 가리키는 매개변수입니다. start와 end는 시작·끝 시각을 담는 지역변수입니다. 10월을 선택하면 10월 1일 0시부터 11월 1일 0시 직전까지 조회합니다. 이렇게 하면 11월 1일 방문이 10월에 섞이지 않습니다.

@Transactional(readOnly = true)는 이 작업이 DB를 읽는 용도라고 Spring에 알려줍니다. Repository가 같은 매장·기간의 고객별 결과를 목록으로 가져옵니다. var는 오른쪽 값을 보고 Java가 변수 타입을 정하게 합니다.

2. 손님별 숫자를 한 줄씩 더해요

```
long revenue = 0;
long visits = 0;
long repeatCustomers = 0;
for (var customer : customers) {
    revenue += customer.getRevenue();
    visits += customer.getVisitCount();
    if (customer.getVisitCount() >= 2) {
        repeatCustomers++;
    }
}
```

for / customer	목록에서 손님별 집계 결과를 한 줄씩 꺼냅니다. customer는 그 한 줄을 가리키는 반복문의 지역변수입니다.
revenue / visits	두 지역변수는 0에서 시작합니다. +=는 지금까지 모은 금액이나 횟수에 이번 손님의 숫자를 더합니다.
repeatCustomers++	같은 달에 두 번 이상 방문한 손님이 나오면 재방문 고객 수를 1 늘립니다. 다섯 번 온 손님도 한 명으로 셉니다.
customers.size()	고객마다 목록에 한 줄씩 있으므로, 줄 수가 방문한 사람 수입니다. 한 사람이 두 번 와도 이 숫자에는 한 명으로 들어갑니다.

03 요일과 시간 그래프

방문 숫자를 알맞은 칸에 넣어요

이 달에 온 손님들을 요일 칸 7개와 시간 칸 24개에 나눠 담으면 막대그래프를 그릴 수 있습니다.

요일은 월요일부터 놓아요

```
long[] weekdayVisits = new long[7];
for (var day : repository.countByWeekday(storeId, start, end)) {
    int mondayBasedIndex = (day.getDayNumber() + 5) % 7;
    weekdayVisits[mondayBasedIndex] = day.getVisitCount();
}
```

long[]은 정수를 순서대로 담는 배열 타입입니다. new long[7]은 값이 0인 칸 7개를 만듭니다. day는 DB에서 받은 요일별 결과 한 줄을 가리키는 지역변수입니다. getDayNumber()로 요일 번호를, getVisitCount()로 방문 건수를 꺼냅니다.

DB는 일요일을 1, 월요일을 2로 셉니다. 화면 배열은 월요일이 첫 칸이므로 번호가 0입니다. (번호 + 5) % 7로 바꾸면 월요일은 0, 일요일은 6이 됩니다. %는 나눗셈의 나머지를 구합니다.

시간은 0시부터 23시까지 놓아요

```
long[] hourlyVisits = new long[24];
for (var hour : repository.countByHour(storeId, start, end)) {
    hourlyVisits[hour.getVisitHour()] = hour.getVisitCount();
}
```

hour는 시간대별 결과 한 줄을 가리키는 지역변수입니다. getVisitHour() 값이 10이면 배열의 10번 칸에 건수를 넣습니다. 오전 10시 10분과 10시 50분 방문은 10시 칸에서 함께 세어 2건이 됩니다. 방문이 없는 칸은 처음 값인 0을 그대로 둡니다.

계산한 값을 결과 상자에 담아요

```
return new Summary(revenue, visits, customers.size(), repeatCustomers,
        weekdayVisits, hourlyVisits, monthlyTrend(storeId, month));
```

new Summary(...)는 요약 결과 객체를 만듭니다. 카드 숫자와 요일·시간 배열, 월별 추이 목록을 인수로 전달합니다. return은 완성한 상자를 호출한 컨트롤러에 돌려줍니다. storeId와 month는 이 서비스가 받는 매개변수이면서 monthlyTrend()를 호출할 때 보내는 인수입니다.

04 6개월 그래프와 결과

달력 여섯 칸과 결과 상자

월별 그래프는 선택한 달까지 6개월을 비교합니다. 방문 기록이 없는 달도 달력에는 자리가 있습니다.

먼저 빈 달을 채우고, DB 결과가 있는 칸을 바꿔요

```
private List<MonthlyPoint> monthlyTrend(long storeId, YearMonth month) {
    MonthlyPoint[] points = new MonthlyPoint[6];
    YearMonth firstMonth = month.minusMonths(points.length - 1);
    for (int index = 0; index < points.length; index++) {
        points[index] = new MonthlyPoint(firstMonth.plusMonths(index).toString(),
                0, 0);
    }
```

monthlyTrend()는 서비스 안에서 쓰는 private 메서드입니다. points는 여섯 달의 배열을 가리키는 지역변수입니다. 10월에서 5개월을 빼면 5월이므로, 5월부터 10월까지 각 칸에 월 이름과 0원·0명을 먼저 넣습니다.

```
var totals = repository.summarizeByMonth(storeId,
        firstMonth.atDay(1).atStartOfDay(), month.plusMonths(1).atDay(1).atStartOfDay());
for (var total : totals) {
    YearMonth visitMonth = YearMonth.of(total.getVisitYear(), total.getVisitMonth());
    int index = (int) firstMonth.until(visitMonth, ChronoUnit.MONTHS);
    points[index] = new MonthlyPoint(visitMonth.toString(),
            total.getRevenue(), total.getCustomerCount());
}
return List.of(points);
```

totals는 DB에서 받은 월별 목록입니다. visitMonth는 결과의 연도와 월을 묶은 객체입니다. until(..., ChronoUnit.MONTHS)는 첫 달부터 몇 달 떨어져 있는지 구합니다. 5월에서 10월은 5칸 뒤라 배열의 5번 칸을 바꿉니다. (int)는 칸 번호에 쓸 정수로 바꾸고, List.of(points)는 순서대로 목록을 만듭니다.

record는 값을 담아 보내는 상자예요

MonthlyPoint	한 달의 이름(String), 결제 합계(long), 고유 방문 고객 수(long)를 담습니다.
Summary	매출, 방문 건수, 방문 고객 수, 재방문 고객 수, 요일·시간 배열, 월별 추이 목록을 담습니다. Java가 revenue() 같은 값 읽기 메서드를 만들어 줍니다.

```
public Long repeatRatePercent() {
    return customers == 0 ? null : Math.round(100.0 * repeatCustomers / customers);
}
```

재방문 고객 수를 방문 고객 수로 나누고 100을 곱합니다. 3명 중 2명이 다시 왔다면 반올림해 67%입니다. 100.0은 소수까지 계산하게 합니다. 방문 고객이 0명이면 null을 돌려주므로 반환 타입은 null을 담을 수 있는 Long입니다.